VEX Notes

Vulnerability exploitability context, SBOM correlation, affected/not affected status logic, and operational VEX notes.

Field	Value
Project	Enterprise DevSecOps Security Lab
Document Owner	Jagriti Banerjee
Document Type	VEX Notes
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, GHCR
Classification	Portfolio / Private Lab Evidence
Status	Final Detailed Spacing Fixed

Document Scope

This report is part of the Enterprise DevSecOps evidence package. It is written for technical reviewers and recruiters who want to understand how artifact trust, SBOM visibility, provenance, VEX notes, and exception governance were implemented and validated in the private lab.

Reviewer Focus Area	What this document proves
Exploitability context	Classifies findings as affected, not_affected, fixed, or under_investigation.
SBOM correlation	Connects component inventory and scanner output to practical risk decisions.
Quality control	Defines evidence expectations for each VEX status.

Evidence Package	Included
Supply chain controls	Cosign signing, Syft SBOM, provenance attestation, Gatekeeper metadata controls
Decision records	Security gate policy, exception rules, VEX status notes, retest expectations
Release quality focus	Artifact integrity, component transparency, vulnerability triage, and audit readiness

Executive Summary

This VEX Notes document explains exploitability status for vulnerabilities and weaknesses in the Enterprise DevSecOps project. VEX helps convert raw scanner output into actionable context: affected findings need remediation, not_affected findings require justification, fixed findings require retest proof, and under_investigation findings require owners and deadlines.

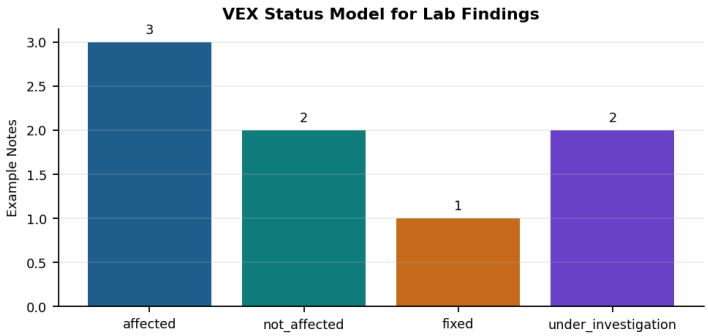

Figure 1 - VEX status model for project findings.

VEX Status Definitions

VEX Status	Meaning	Required Evidence
affected	The vulnerability is relevant to the product or lab artifact.	Reachable package, vulnerable code path, scanner result, PoC, or deployment exposure.
not_affected	The component exists but the vulnerability is not exploitable in this context.	Justification such as vulnerable function not used, component not reachable, or feature disabled.
fixed	The issue has been remediated in the artifact or deployment.	Patch commit, updated package version, scanner clean result, or retest proof.
under_investigation	Exploitability is not yet determined.	Open analysis task, owner, expiry date, additional tests required.

Decision Tree for VEX Status

Question	If Yes	If No
Is the vulnerable package present in the SBOM?	Continue exploitability analysis.	Status may be not_affected with inventory evidence.
Is the vulnerable code path reachable?	Status likely affected.	Status may be not_affected with reachability justification.
Has remediation been applied and retested?	Status fixed.	Continue affected or under_investigation.
Is there enough evidence to decide?	Record status with evidence.	Use under_investigation with owner and due date.

Project VEX Notes

VEX ID	Finding	Status	Evidence and Rationale
VEX-001	SQL injection in /user	affected	Endpoint is intentionally vulnerable and exploit proof returns multiple users.
VEX-002	Command injection in /ping	affected	User input reaches shell=True command construction.
VEX-003	Unsafe template rendering	affected	User input is inserted into render_template_string.
VEX-004	Outdated Python dependencies	under_investigation	Scanner evidence exists; package usage and exploit paths require triage.

VEX ID	Finding	Status	Evidence and Rationale
VEX-005	Privileged pod with hostPath	fixed	Pod Security restricted enforcement blocks the dangerous pod.
VEX-006	Overprivileged ServiceAccount	fixed	cluster-admin binding removed; least privilege role applied.
VEX-007	Missing supply chain labels	fixed	Gatekeeper blocks non-compliant deployments and allows compliant ones.

SBOM, Scanner, Exception, and VEX Relationship

Document / Artifact	Primary Purpose	How It Connects to VEX
SBOM	Lists components and versions.	Shows whether vulnerable component exists.
Scanner output	Identifies possible CVEs/misconfigurations.	Creates the finding requiring VEX context.
VEX note	Explains exploitability status.	Classifies affected, not_affected, fixed, or under_investigation.
Exception register	Approves temporary risk acceptance.	Uses VEX context to support or reject exception requests.
Retest proof	Validates closure.	Supports fixed status and exception closure.

VEX Note Quality Checklist

- Every note must name the artifact and image digest when possible.
- Every not_affected note must include a justification or impact statement.
- Every affected note must link to remediation plan or lab-only teaching objective.
- Every fixed note must include retest evidence.
- Every under_investigation note must have owner, due date, and next analysis step.
- VEX notes must be updated whenever the image, dependency tree, or deployment context changes.

References

- CISA VEX minimum requirements: https://www.cisa.gov/sites/default/files/2023-04/minimum-requirements-for-vex-508c.pdf
- OpenVEX specification: https://github.com/openvex/spec/blob/main/OPENVEX-SPEC.md
- SLSA specification: https://slsa.dev/spec/v1.2/
- SLSA provenance specification: https://slsa.dev/spec/v1.0/provenance
- Sigstore Cosign project: https://github.com/sigstore/cosign
- Cosign specifications and attestation support: https://docs.sigstore.dev/cosign/system_config/specifications/
- Sigstore project overview: https://www.sigstore.dev/